

Министерство образования Новосибирской области
ГБПОУ НСО «Новосибирский авиационный технический колледж имени Б.С. Галуцака»

УТВЕРЖДАЮ
Председатель цикловой
комиссии специальности
09.02.07
_____ А.А. Терехова
Протокол № ____ от
« ____ » _____ 20 ____ г

УТВЕРЖДАЮ
Председатель цикловой
комиссии специальности
09.02.07
_____ А.А. Терехова
Протокол № ____ от
« ____ » _____ 20 ____ г

УТВЕРЖДАЮ
Председатель цикловой
комиссии специальности
09.02.07
_____ А.А. Терехова
Протокол № ____ от
« ____ » _____ 20 ____ г

**Разработка интерфейса в React функциональные компоненты, хуки useState/useEffect,
маршрутизация**

Методические указания к практическому занятию №10

Учебная дисциплина: ОПВ.15 Проектирование, разработка и оптимизация веб-приложений
Специальность: 09.02.07 Информационные системы и программирование

Разработал:
Зиберева А.А.

1 Цели

1.1 В ходе выполнения работы студенты осваивают:

1.1.1 Общие компетенции:

ОК 01 — Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам

ОК 02 — Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

ОК 09 — Использовать информационные технологии в профессиональной деятельности

1.1.2 Профессиональные компетенции:

ПК 9.2 — Разрабатывать веб-приложение в соответствии с техническим заданием.

ПК 9.3 — Разрабатывать интерфейс пользователя веб-приложений в соответствии с техническим заданием

1.2 В результате выполнения практического задания студенты:

1.2.1 Приобретают практический опыт:

- инициализации и настройки современного React-проекта с использованием инструмента сборки Vite и библиотеки React Router;

- разработки пользовательского интерфейса на основе функциональных компонентов и типизации данных с помощью TypeScript;

- реализации навигации в одностраничном приложении (SPA) без перезагрузки страниц;

- управления глобальным состоянием приложения (корзиной товаров) через механизм React Context и кастомные хуки;

- стилизации интерфейса с применением современного CSS-фреймворка Tailwind CSS.

1.2.2 Осваивают навыки:

- создания переиспользуемых UI-компонентов (кнопки, модальные окна, карточки товаров);

- работы с базовыми хуками React (useState, useMemo, useContext) для управления логикой и оптимизации рендеринга;

- проектирования структуры данных веб-приложения с использованием интерфейсов TypeScript;

- организации файловой структуры проекта (папки components, hooks, data, types) для масштабируемой разработки;

- динамической подгрузки медиа-контента (изображений) и работы с относительными путями в ESM-модулях.

1.2.3 Усваивают знания:

- основных принципов компонентного подхода и жизненного цикла функциональных компонентов;

- преимуществ использования фреймворков и библиотек маршрутизации для создания современных веб-приложений;

- роли и преимуществ TypeScript в предотвращении ошибок на этапе разработки;

- архитектуры «Провайдер — Потребитель» для управления общими данными в React;

- подходов к созданию адаптивного и интерактивного дизайна с использованием утилит Tailwind CSS.

2 Оборудование, аппаратура, материалы, инструменты, документация, дидактическое обеспечение

- персональный компьютер с установленной ОС (Windows, macOS, Linux);

- среда выполнения Node.js (версия 18.x или выше) и менеджер пакетов npm;

- интегрированная среда разработки (IDE) VS Code с расширениями для работы с TypeScript и Tailwind CSS;

- веб-браузер (Google Chrome, Firefox или аналогичный) с инструментами разработчика;

- доступ к сети Интернет для инициализации проекта через npx и загрузки зависимостей;

- библиотеки и инструменты: React 19, React Router v7, Vite, Tailwind CSS v4, TypeScript;

- дидактический материал: методические указания к практической работе №10, доступ к ресурсам react.dev, Google Fonts, Unsplash и UI Colors.

3 Форма организации – индивидуальная (групповая в случае нехватки рабочих мест).

4 Инструктаж

4.1 Время выполнения работы – 6 академических часов (270 минут)

4.2 Работа состоит из заданий, приведенных ниже в пункте 9

4.3 При выполнении работы следует пользоваться методическими указаниями, приведенными в приложении А, либо конспектом соответствующей темы и выполнить задания

5 Порядок выполнения

5.1 Ознакомиться с теоретическими сведениями о практической работе

5.2 Ознакомиться с формой отчета по практическому занятию

5.3 Ознакомиться с методическими указаниями, приведенными в приложении А

5.4 Выполнить работу, оформить отчет, предоставить оба на проверку преподавателю

6 Методические рекомендации

6.1 Методические рекомендации приведены в приложении А

7 Форма отчета

Отчет должен содержать:

- номер и название практического занятия;
- фамилию, номер группы и вариант студента;
- выполненные задания практической работы.

8 Критерии оценок

- «Отлично» - при условии выполнения всех заданий;
- «Хорошо» - при условии выполнения всех заданий, но допущены неточности в их решении;
- «Удовлетворительно» - допущены ошибки при выполнении заданий, отчет оформлен частично,
- «Неудовлетворительно» - если практические задания не выполнены или допущены ошибки, влияющие на качество выполненной работы, отчет по работе отсутствует.

9 Содержание заданий

Задание

Необходимо выполнить практическое задание, направленное на проектирование и разработку многостраничного пользовательского интерфейса веб-приложения для ресторана с использованием библиотеки React, настройку клиентской маршрутизации через React Router, создание системы управления состоянием корзины с помощью React Context API и стилизацию компонентов с применением фреймворка Tailwind CSS.

ПРИЛОЖЕНИЕ А

Инструкции к выполнению заданий

Шаги выполнения работы

1 Начало работы

Для того, чтобы начать проект, необходимо его инициализировать с помощью подготовленной для этого команды. На сайте react.dev можно узнать о том, как создать новое приложение на React. Ранее это можно было сделать с помощью команды `npm npx create-react-app@latest название-приложения`. Теперь данная команда устарела и разработчики React (команда Meta) теперь рекомендуют создавать новые приложения с использованием полнофункциональных фреймворков (таких как Next.js, Remix или Gatsby), потому что сам по себе React — это только библиотека для отрисовки пользовательского интерфейса (UI). Для создания современного приложения требуются дополнительные возможности (маршрутизация, серверный рендеринг, оптимизация данных), которые фреймворки предоставляют из коробки. Если мы решим сделать приложение на React «from scratch», то придется писать свой собственный фреймворк.

Поэтому, для данной практической работы мы будем использовать уже существующее решение — React Router.

Команда для инициализации проекта — `npx create-react-router@latest restaurant-app` (последнее значение такое, т. к. мы будем разрабатывать интерфейс приложения для ресторана. Данное приложение не будет иметь базы данных и реальной оплаты — в данной практической работе нашец целью является разработка интерфейса предполагаемого ресторана, а не прямая разработка полного приложения с привязкой к базе данных.

Поговорим про самые важные инструменты для нашего приложения.

React Router — это стандартная библиотека для маршрутизации (routing) в React-приложениях, позволяющая переключаться между разными представлениями (компонентами) без перезагрузки страницы. Она синхронизирует URL-адрес браузера с отображаемым контентом, превращая SPA (одностраничные приложения) в многостраничные. Т.е. мы не будем обновлять страницу для перехода на другую — все будет обновляться сразу же, мгновенно.

Также мы будем использовать Vite — современный, чрезвычайно быстрый инструмент сборки и сервер разработки для фронтенд-проектов. Он создан Эваном Ю (разработчиком Vue.js) для ускорения процесса разработки за счет использования нативных ES-модулей в браузере, обеспечивая мгновенную горячую замену модулей (HMR) без перезагрузки страницы. Vite нужен, чтобы сделать процесс разработки React-приложений комфортным, убрав долгое ожидание сборки и перекомпиляции.

Конечно, мы будем также использовать Tailwind CSS как альтернативу обычному CSS и HTML. Данный фреймворк также идет встроенным в шаблон, который мы будем использовать.

Вернемся к использованию команды: нам необходимо открыть CMD в любой папке, в которой нам будет удобно создать проект (т.к. при запуске команды создастся новая папка с именем проекта, который мы задали) и написать туда команду, упомянутую ранее.

На рисунке 1.1 представлена реакция терминала на запуск команды — мы обязательно соглашаемся на установку зависимостей через npm и на инициализирование нового репозитория Git.

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\User>npx create-react-router@latest restaurant-app

  create-react-router v7.15.0
  ■ Directory: Using restaurant-app as project directory
  ■ Using default template See https://github.com/remix-run/react-router-templates for more
  ✓ Template copied

  git Initialize a new git repository?
  Yes

  deps Install dependencies with npm?
  Yes

  ✓ Dependencies installed
  ✓ Git initialized

  done That's it!

  Enter your project directory using cd .\restaurant-app
  Check out README.md for development and deploy instructions.

  Join the community at https://rmx.as/discord
```

Рисунок 1.1 — Настройка проекта

После чего необходимо открыть новую папку проекта через удобный редактор кода и рассмотреть представленную для нас структуру (она будет изменена в будущем). Таким образом, структура проекта, на данном этапе, будет выглядеть так, как на рисунке 1.2.

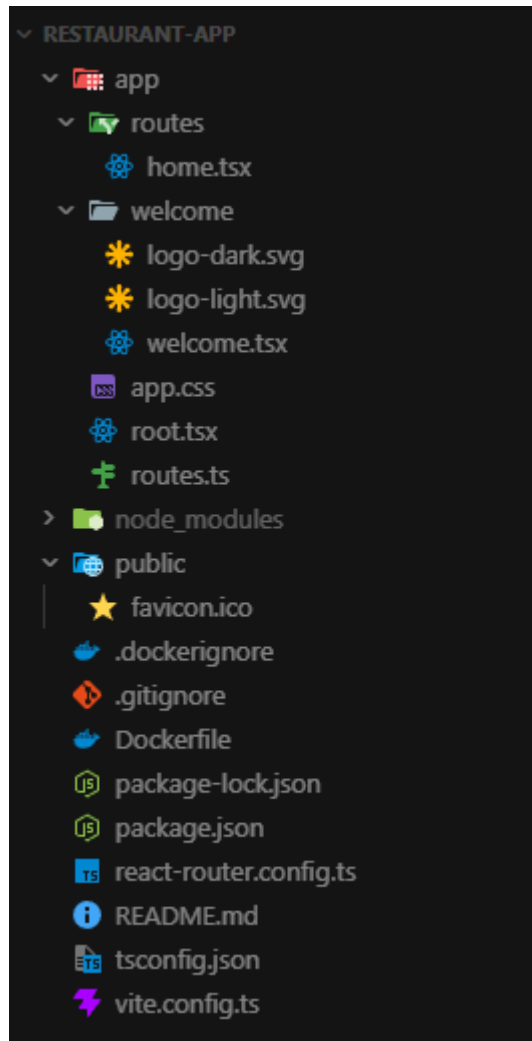


Рисунок 1.2 — Шаблонная структура проекта

- папка welcome нам не нужна, мы можем от нее избавиться (не бойтесь предупреждений или ошибок, которые возникнут в шаблонных файлах после удаления);
- папка public содержит favicon.ico (иконка приложения на вкладке); ее нельзя удалять, но можно будет заменить позже, если хочется;
- все остальные файлы, лежащие в корне проекта — конфигурационные и просто нужны для работы инструментов (к примеру, Vite или Docker);
- папка app в будущем будет содержать все нужные файлы и компоненты приложения;
- папка routes и файл routes отвечают за маршрутизацию: папка содержит компоненты страниц, а файл выступает центральным узлом конфигурации для маршрутов;
- app.css — файл для общей глобальной настройки стилей для Tailwind CSS в проекте;
- root.tsx — файл-корневой компонент (root layout), который служит оберткой для всего приложения.

В остальном наша структура будет изменяться в ходе разработки приложения. Но, напоследок, можно взглянуть на package.json на рисунке 1.3. В нем уже содержится вся нужная информация для нашего проекта, включая название, скрипты и установленные зависимости.

```

package.json > ...
1  {
2    "name": "restaurant-app",
3    "private": true,
4    "type": "module",
5    ▶ Debug
6    "scripts": {
7      "build": "react-router build",
8      "dev": "react-router dev",
9      "start": "react-router-serve ./build/server/index.js",
10     "typecheck": "react-router typegen && tsc"
11   },
12   "dependencies": {
13     "@react-router/node": "7.15.0",
14     "@react-router/serve": "7.15.0",
15     "isbot": "^5.1.36",
16     "react": "^19.2.5",
17     "react-dom": "^19.2.5",
18     "react-router": "7.15.0"
19   },
20   "devDependencies": {
21     "@react-router/dev": "7.15.0",
22     "@tailwindcss/vite": "^4.2.2",
23     "@types/node": "^22",
24     "@types/react": "^19.2.14",
25     "@types/react-dom": "^19.2.3",
26     "tailwindcss": "^4.2.2",
27     "typescript": "^5.9.3",
28     "vite": "^8.0.3"
29   }
30 }

```

Рисунок 1.3 — package.json

Здесь установлены следующие зависимости:

- @react-router/node — адаптер для запуска приложений React Router (v7) в среде Node.js (например, для работы с HTTP-запросами и ответами на сервере);
- @react-router/serve — готовый легковесный сервер для запуска приложения в продакшене без необходимости писать свой серверный код;
- isbot — библиотека для определения того, является ли посетитель сайта поисковым роботом (нужно для корректной индексации и SEO);
- react — основная библиотека для создания пользовательских интерфейсов;
- react-dom — пакет для отрисовки компонентов React в браузере (DOM);
- react-router — ядро библиотеки для управления навигацией и маршрутизацией в приложении;
- @react-router/dev — инструменты разработчика для React Router (плагины для Vite, генерация типов и CLI-команды);
- @tailwindcss/vite — плагин для интеграции Tailwind CSS версии 4 напрямую в сборщик Vite для максимальной скорости работы;
- @types/node — описание типов TypeScript для работы с окружением Node.js;
- @types/react — типы TypeScript для библиотеки React;
- @types/react-dom — типы TypeScript для библиотеки React DOM;
- tailwindcss — сам фреймворк Tailwind CSS для стилизации через готовые CSS-классы;

- typescript — язык программирования и компилятор, добавляющий статическую типизацию в JavaScript;
- vite — современный и быстрый инструмент для сборки фронтенда и запуска сервера разработки.

Как вы могли заметить, мы будем писать наш проект с использованием TypeScript. Почему? Основные преимущества включают раннее обнаружение ошибок до запуска, улучшенную читаемость, удобный рефакторинг и мощную автодополнение в IDE. Это делает его стандартом для крупных и командных веб-разработок. Также типизация позволяет выявлять опечатки, неправильные типы данных и другие распространенные баги на этапе компиляции, а не во время работы приложения, что значительно сокращает время разработки и делает ее более комфортной.

Теперь перейдем к непосредственной разработке приложения, но начнем с того, чтобы подобрать нужный стиль под него и необходимые картинки для нашего меню.

2 Подбор шрифтов, медиа-контента и цветовой палитры для интерфейса

Данная практическая работа будет немного творческой в плане составления своего меню, добавления картинок, подбора цветовой палитры и шрифта для интерфейса. То есть, делать такие же цвета, картинки, меню и другие вещи — не обязательно такими, как в методических рекомендациях. Но, важно написать такой же код, чтобы понять, как все сделать правильно.

Для подбора цветов лучше всего использовать ресурс <https://uicolors.app/>. Он помогает подобрать лаконичную палитру цветов для приложения и сразу же показывает ее на примерном интерфейсе (т.е. как будут выглядеть различные компоненты с данной палитрой цветов. На рисунке 2.1 изображен пример как это выглядит.

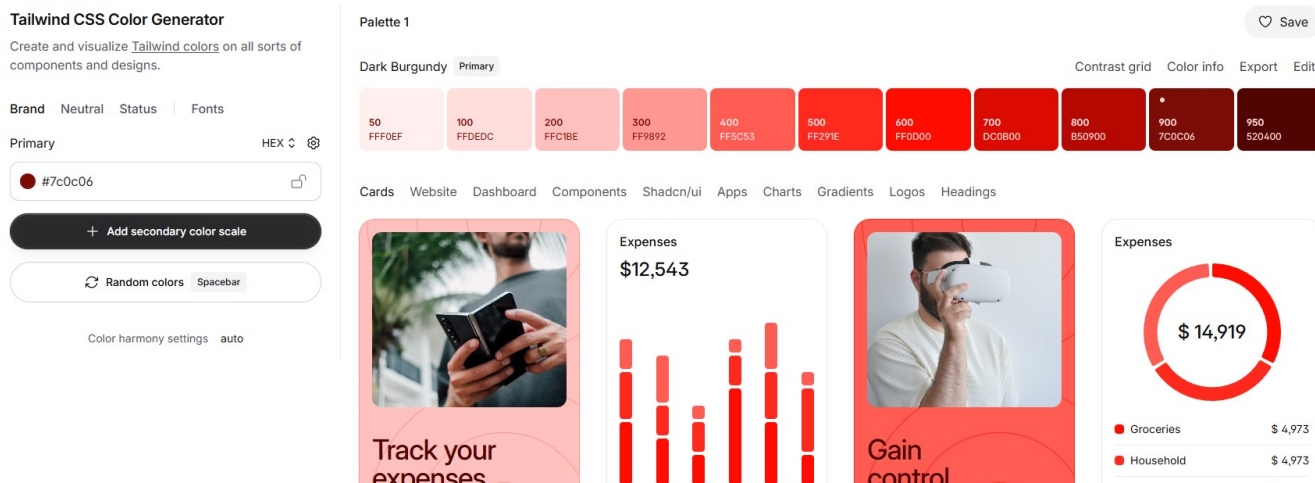


Рисунок 2.1 — Пример сгенерированной палитры цветов

Шрифты, можно подобрать через Google Fonts, но только подобрать те, которые могут быть совместимы с кириллицей (так как не все шрифты хорошо работают с русскими буквами).

Картинки для блюд можно скачать с сайта <https://unsplash.com/>. Он содержит огромное количество различных бесплатных ассетов на любой вкус.

На рисунке 2.2 представлен модифицированный файл app.css, а на рисунке 2.3 — папка с картинками для меню и других целей.

```

app > app.css > ...
...
1  @import url('https://fonts.googleapis.com/css2?family=Manrope:wght@200..800&display=swap');
2  @import "tailwindcss";
3
4  html,
5  body {
6      background-color: #f3f7ee;
7      color: #323f28;
8      font-family:
9          "Manrope",
10         system-ui,
11         -apple-system,
12         "Segoe UI",
13         Roboto,
14         "Helvetica Neue",
15         Arial,
16         "Noto Sans",
17         sans-serif;
18  }
19
20  @media (prefers-color-scheme: dark) {
21      html,
22      body {
23          color-scheme: dark;
24      }
25  }
26
27  @theme {
28      --color-tom-thumb-50: #f3f7ee;
29      --color-tom-thumb-100: #e4ecdb;
30      --color-tom-thumb-200: #ccdcba;
31      --color-tom-thumb-400: #8eae6d;
32      --color-tom-thumb-600: #56733d;
33      --color-tom-thumb-700: #485f35;
34      --color-tom-thumb-800: #38482c;
35      --color-tom-thumb-900: #323f28;
36      --color-tom-thumb-950: #182112;
37  }

```

Рисунок 2.2 — app.css

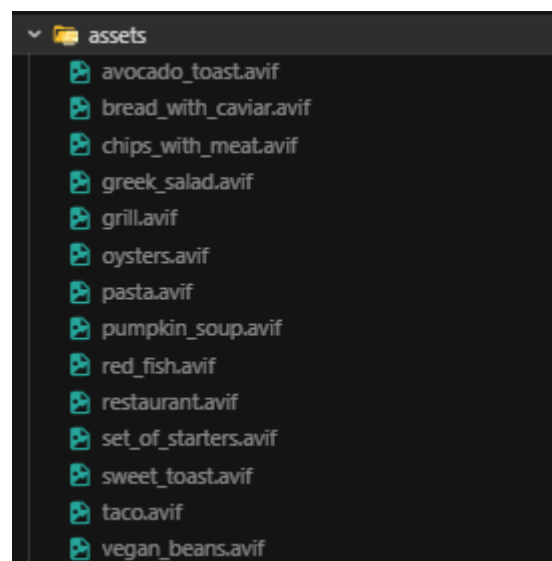


Рисунок 2.3 — assets

Папка assets — одна из первых папок, которые мы создаем в папке app. К ней мы будем обращаться при создании имитации набора данных для меню.

3 Создание интерфейсов

Первым делом нам нужно определить строгие типы для данных, которые будут в нашем приложении.

Создадим папку `types` в папке `app` и создадим в ней файл `index.ts`. На рисунке 3.1 показано, как это должно выглядеть.

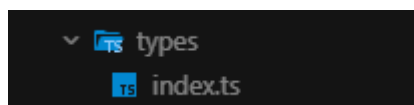


Рисунок 3.1 — types

В файле `index.ts` мы храним интерфейсы. Интерфейсы в TypeScript (`interface`) — это мощный инструмент для описания структуры объектов, классов и API, позволяющий строго типизировать данные без создания реальных объектов. Файл `index.ts` часто используется как точка входа для экспорта этих интерфейсов, чтобы упорядочить структуру проекта. На рисунке 3.2 показаны интерфейсы, которые мы определяем для нашего приложения: элемент меню, элемент корзины, информация о ресторане и информация о заказе.

```
app > types > TS index.ts > ...  
  
...  
1  export interface MenuItem {  
2    id: number;  
3    name: string;  
4    description: string;  
5    price: number;  
6    category: "Закуски" | "Основные блюда" | "Десерты" | "Напитки";  
7    image: string;  
8  }  
9  
...  
10 export interface CartItem {  
11   menuItem: MenuItem;  
12   quantity: number;  
13 }  
14  
...  
15 export interface RestaurantInfo {  
16   name: string;  
17   address: string;  
18   phone: string;  
19   workHours: string;  
20 }  
21  
...  
22 export interface OrderData {  
23   items: CartItem[];  
24   total: number;  
25   customerName: string;  
26 }
```

Рисунок 3.2 — index.ts

Почти для всех указаны простые типы данных, такие как число, строка и т. д., но есть и те, в которых мы указываем, к примеру, массив данных (массив блюд в корзине), другой интерфейс или перечисление (строковый литеральный тип или тип-объединение) для категорий блюд.

Теперь нужно создать данные, которые будут у нас в меню. Так как картинки есть — остается только создать массив данных, который будет использоваться в приложении. Здесь тоже все достаточно просто: создаем папку data в папке app и внутри создаем два файла — restaurant.ts и menu.ts. Первый будем выполнять роль информации о ресторане, которую можно будет потом использовать в приложении, а второй — содержать список блюд и их характеристик.

На рисунке 3.3 изображена папка, а на рисунке 3.4 — часть файла с меню.

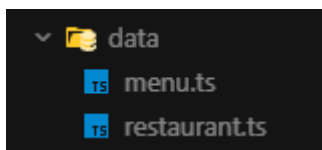


Рисунок 3.3 — data

```
app > data > menu.ts > ...
You, 15 minutes ago | 1 author (You)
1 import type { MenuItem } from "~/types";
2
3 export const menuData: MenuItem[] = [
4   {
5     id: 1,
6     name: "Тост с авокадо и яйцом",
7     description: "Тост с авокадо и яйцом с листиками базилика и оливковым маслом",
8     price: 250,
9     category: "Закуски",
10    image: new URL("../assets/avocado_toast.avif", import.meta.url).href,
11  },
12  {
13    id: 2,
14    name: "Багет с черной икрой",
15    description: "Багет с черной икрой и кусочками сот с медом",
16    price: 250,
17    category: "Закуски",
18    image: new URL("../assets/bread_with_caviar.avif", import.meta.url).href,
19  },
20  {
21    id: 3,
22    name: "Чипсы и мясная нарезка",
23    description: "Чипсы и мясная нарезка с сыром и оливками",
24    price: 250,
25    category: "Закуски",
26    image: new URL("../assets/chips_with_meat.avif", import.meta.url).href,
27  },
28  {
29    id: 4,
30    name: "Греческий салат",
31    description: "Греческий салат с оливками, помидорами, красным болгарским перцем и орегано",
32    price: 250,
33    category: "Закуски",
34    image: new URL("../assets/greek_salad.avif", import.meta.url).href,
35  },
36  {
37    id: 5
```

Рисунок 3.4 — menu.ts

Принцип такой:

- мы импортируем типы (наши интерфейсы) внутри данных через алиас; в TypeScript (и современной веб-разработке в целом) использование алиасов путей (path aliases) значительно лучше, чем относительные пути через точки (../..), особенно в средних и крупных проектах; можно не переживать из-за нагромождений точек для правильного пути к файлу или, к примеру, при переносе файла, можно не менять импорт;

- создаем переменную, в которой будут храниться блюда и указываем ей тип; данное указание типа говорит системе, что в этой переменной может храниться только массив объектов, соответствующих структуре MenuItem;

- присваиваем ей массив, в котором будут храниться наши структуры блюд.

Не забываем, что массив закрывается квадратными скобками слева и справа.

Самое главное, что нужно тут упомянуть — обращение к пути картинок. Такие строки, как на рисунке, нужны для динамического получения правильного пути к файлу изображения после того, как проект будет собран (скомпилирован).

`new URL('...', import.meta.url)` создает объект ссылки, второй аргумент `import.meta.url` — это встроенная переменная, которая указывает на текущий адрес самого файла, а `href` извлекает итоговую строковую ссылку, которую можно подставить в тег ``.

При сборке сайта для публикации (production) сборщики часто переименовывают файлы, чтобы избежать проблем с кэшем. Структура папок на вашем компьютере и на сервере может отличаться. Конструкция `new URL` гарантирует, что ссылка не «сломается», где бы ни находился файл.

Таким образом мы создаем то количество блюд, которое нам необходимо.

На рисунке 3.5 представлен пример информации о ресторане. Принцип — абсолютно тот же.

```
app > data > TS restaurant.ts > ...  
...  
1  import type { RestaurantInfo } from "~/types";  
2  
3  export const restaurantInfo: RestaurantInfo = {  
4    name: "НАТК",  
5    address: "ул. Галуцака, 72",  
6    phone: "+7 (999) 123-45-67",  
7    workHours: "Пн-Вс: 12:00 - 23:00",  
8  };
```

Рисунок 3.5 — restaurant.ts

4 Создание компонентов

Вспомним, что такое компоненты в React. React-компоненты — это независимые, переиспользуемые блоки кода, составляющие пользовательский интерфейс (UI) в React-приложении. Они работают как функции JavaScript, принимая входные данные (props) и возвращая HTML-подобную структуру JSX или TSX, описывающую отображение на экране. Компоненты позволяют разбивать сложные интерфейсы на простые, управляемые части.

Помним, что компоненты у нас могут быть только функциональные.

Создадим папку components в папке app. Внутри данной папки у нас будут храниться части UI — кнопка и модальное окно, которое будет появляться поверх интерфейса, когда пользователь оформит заказ и «оплатит» его, и остальные компоненты — подвал, шапка, карточка меню и элемент корзины. Первые компоненты будут храниться внутри папки ui, а все остальные — в корне папки components.

На рисунке 4.1 изображена структура данной папки.

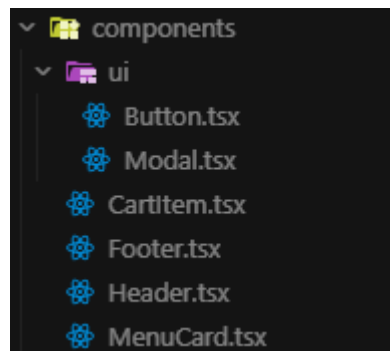


Рисунок 4.1 — components

Первый компонент — Button.tsx. Рассмотрим его код на рисунке 4.2 и разберем его подробнее.

```
app > components > ui > Button.tsx > ...  
You, 16 hours ago | 1 author (You)  
1 import type { ButtonHTMLAttributes, ReactNode } from "react";  
2  
You, 16 hours ago | 1 author (You)  
3 interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {  
4   children: ReactNode;  
5   variant?: "primary" | "secondary";  
6 }  
7  
8 export default function Button({  
9   children,  
10  variant = "primary",  
11  className = "",  
12  ...props  
13 }: ButtonProps) {  
14  const baseClass = "px-6 py-3 rounded-xl font-medium transition-colors disabled:opacity-50";  
15  const variants = {  
16    primary: "bg-tom-thumb-600 text-white hover:bg-tom-thumb-700",  
17    secondary: "bg-stone-200 text-stone-700 hover:bg-stone-300",  
18  };  
19  
20  return (  
21    <button  
22      className={` ${baseClass} ${variants[variant]} ${className}`}  
23      {...props}  
24    >  
25      {children}  
26    </button>  
27  );  
28 }
```

Рисунок 4.2 — Button.tsx

В начале файла происходит импорт типов `ButtonHTMLAttributes` и `ReactNode` из библиотеки `React`. Тип `ReactNode` используется для описания содержимого кнопки. Это означает, что внутрь кнопки можно передавать текст, иконки, другие элементы или даже несколько элементов одновременно. Тип `ButtonHTMLAttributes<HTMLButtonElement>` нужен для того, чтобы компонент автоматически поддерживал все стандартные HTML-атрибуты кнопки, например `onClick`, `disabled`, `type` и другие. Благодаря этому компонент работает как обычная HTML-кнопка, но при этом остается более удобным и гибким.

Далее создается интерфейс `ButtonProps`, который описывает свойства компонента. Интерфейс наследует все стандартные параметры кнопки. Помимо этого добавляется собственное свойство `variant`. Оно определяет внешний вид кнопки и может принимать только два значения: `"primary"` или `"secondary"`. Также обязательным свойством является `children`, так как именно через него передается содержимое кнопки.

После описания интерфейса создается сам компонент `Button`. Он экспортируется по умолчанию с помощью `export default`, чтобы компонент можно было импортировать в другие файлы проекта. В параметрах функции используется деструктуризация объекта свойств. Из `props` сразу извлекаются `children`, `variant`, `className`, а остальные параметры собираются в объект `...props`. Значение `variant = "primary"` задается по умолчанию.

Внутри компонента создается переменная `baseClass`. В ней хранятся базовые CSS-классы, которые применяются ко всем кнопкам независимо от их типа. Здесь задаются внутренние отступы, скругление углов, толщина шрифта, плавность анимации изменения цвета и прозрачность для неактивной кнопки. Создается объект `variants`, который содержит варианты оформления кнопки. Вариант `primary` задает зеленый фон, белый текст и более темный оттенок при наведении курсора. Вариант `secondary` использует светлый фон и темный текст.

В блоке `return` компонент возвращает JSX-разметку кнопки. В атрибут `className` передается строка, собранная с помощью шаблонных литералов. Здесь объединяются базовые стили, стили выбранного варианта и дополнительные классы, которые могут быть переданы извне через `className`. Конструкция `{...props}` передает все оставшиеся свойства непосредственно в HTML-элемент `<button>`. Благодаря этому компонент поддерживает стандартное поведение HTML-кнопки без необходимости вручную перечислять все возможные атрибуты.

Внутри тега `<button>` выводится `{children}` — содержимое кнопки, которое было передано при использовании компонента.

Посмотрим на `Modal.tsx`. На рисунке 4.3 показан код данного компонента.

```

app > components > ui > Modal.tsx > ...
You, 16 hours ago | 1 author (You)
1  import type { ReactNode } from "react";
2
3  You, 16 hours ago | 1 author (You)
4  interface ModalProps {
5      isOpen: boolean;
6      onClose: () => void;
7      children: ReactNode;
8      title: string;
9  }
10
11  export default function Modal({ isOpen, onClose, children, title }: ModalProps) {
12      if (!isOpen) return null;
13
14      return (
15          <div className="fixed inset-0 z-50 flex items-center justify-center">
16              <div
17                  className="absolute inset-0 bg-black/50"
18                  onClick={onClose}
19              />
20
21              <div className="relative bg-white rounded-2xl p-8 max-w-lg w-full mx-4 shadow-2xl">
22                  <div className="flex justify-between items-center mb-4">
23                      <h2 className="text-xl font-bold">{title}</h2>
24                      <button
25                          onClick={onClose}
26                          className="text-stone-400 hover:text-stone-600 text-2xl leading-none"
27                      >
28                          ✕
29                      </button>
30                  </div>
31                  {children}
32              </div>
33          </div>
34      );
35  }

```

Рисунок 4.3 — Modal.tsx

Принцип здесь примерно такой же. Как и в предыдущем компоненте, в начале файла импортируется тип `ReactNode`, который позволяет передавать внутрь модального окна любое содержимое: текст, формы, кнопки или другие React-компоненты. Через интерфейс `ModalProps` описываются свойства компонента. Здесь используются четыре параметра: `isOpen`, `onClose`, `children` и `title`.

Свойство `isOpen` имеет тип `boolean` и отвечает за отображение модального окна. Если значение равно `true`, окно отображается на экране, если `false` — компонент скрывается. Свойство `onClose` содержит функцию, которая вызывается при закрытии окна. Свойство `title` отвечает за заголовок окна, а `children`, как и в предыдущем компоненте, используется для передачи внутреннего содержимого.

Главное отличие здесь — условный рендеринг. Проверяется, открыто ли модальное окно. Если `isOpen` равно `false`, компонент ничего не отображает. Возвращаемое значение `null` в React означает отсутствие рендера.

После проверки в коде мы возвращаем разметку окна: первый блок растягивается на весь экран благодаря классу `fixed inset-0`. Класс `z-50` задает высокий уровень слоя, с помощью `flex`, `items-center` и `justify-center` содержимое выравнивается по центру экрана. Следующий блок занимает весь экран и имеет полупрозрачный черный цвет. Обработчик `onClick={onClose}` позволяет закрыть окно при нажатии на затемненную область.

После этого создается само модальное окно. Для него задаются белый фон, скругленные углы, внутренние отступы, ограничение ширины и тень. Заголовок выводится через `{title}`, а кнопка содержит символ крестика. При нажатии вызывается функция `onClose`, что закрывает

модальное окно. В конце компонента выводится `{children}` — содержимое, переданное внутрь модального окна.

Мы закончили создавать небольшие компоненты, перейдем к компонентам, которые нам необходимы для основного экрана и основных элементов интерфейса.

Рассмотрим подвал и шапку, которые будут располагаться на интерфейсе всегда, без изменений. На рисунках 4.4 и 4.5 представлены коды обоих компонентов. По своей сути, они достаточно простые.

```
app > components > Footer.tsx > ...  
...  
1  import { restaurantInfo } from "~/data/restaurant";  
2  
3  export default function Footer() {  
4    return (  
5      <footer className="bg-tom-thumb-950 text-tom-thumb-100 py-8 mt-12">  
6        <div className="max-w-6xl mx-auto px-4 text-center">  
7          <p className="text-lg font-bold text-white mb-2">{restaurantInfo.name}</p>  
8          <p>{restaurantInfo.address}</p>  
9          <p>{restaurantInfo.phone}</p>  
10         <p>{restaurantInfo.workHours}</p>  
11        </div>  
12      </footer>  
13    );  
14  }
```

Рисунок 4.4 — Footer.tsx

```
app > components > Header.tsx > ...  
...  
1  import { Link, NavLink } from "react-router";  
2  
3  export default function Header() {  
4    return (  
5      <header className="bg-tom-thumb-800 text-white shadow-md">  
6        <nav className="max-w-6xl mx-auto px-4 py-4 flex justify-between items-center">  
7          <Link to="/" className="text-2xl font-bold">  
8            HATK  
9          </Link>  
10         <div className="flex gap-8 text-lg">  
11           <NavLink  
12             to="/"  
13             className={({ isActive }) => isActive ? "text-tom-thumb-100" : "text-white hover:text-tom-thumb-200"}  
14           >  
15             Главная  
16           </NavLink>  
17           <NavLink  
18             to="/menu"  
19             className={({ isActive }) => isActive ? "text-tom-thumb-100" : "text-white hover:text-tom-thumb-200"}  
20           >  
21             Меню  
22           </NavLink>  
23           <NavLink  
24             to="/cart"  
25             className={({ isActive }) => isActive ? "text-tom-thumb-100" : "text-white hover:text-tom-thumb-200"}  
26           >  
27             Корзина  
28           </NavLink>  
29           <NavLink  
30             to="/about"  
31             className={({ isActive }) => isActive ? "text-tom-thumb-100" : "text-white hover:text-tom-thumb-200"}  
32           >  
33             О нас  
34           </NavLink>  
35         </div>  
36       </nav>  
37     </header>  
38   );  
39 }
```

Рисунок 4.5 — Header.tsx

Данный компонент содержит дополнительную информацию о компании, контактные данные, адрес, режим работы и другие сведения, которые должны отображаться на всех страницах приложения.

В начале файла импортируется объект `restaurantInfo` из директории `data`. В отличие от предыдущих компонентов, здесь данные не прописываются вручную внутри JSX-разметки, а берутся из отдельного файла. Компонент `Footer` представляет собой обычную функциональную React-компоненту, которая возвращает JSX-разметку. Основным контейнером — тег `<footer>`. Для него задаются стили фона, цвета текста, внутренние отступы и внешний отступ сверху. Это визуально отделяет футер от основного содержимого страницы.

Внутри располагается дополнительный контейнер с ограничением максимальной ширины и центрированием содержимого. Такой подход уже использовался в предыдущих компонентах и помогает поддерживать единый стиль интерфейса.

Данные выводятся через выражения в фигурных скобках. React подставляет значения свойств объекта непосредственно в HTML-разметку. Аналогичным образом отображаются адрес, телефон и режим работы.

`Header.tsx` реализует верхнюю навигационную панель сайта. Такой компонент используется для перемещения пользователя между страницами приложения без полной перезагрузки браузера. В начале файла импортируются компоненты `Link` и `NavLink` из библиотеки `react-router`. Компонент `Link` используется для обычной навигации внутри React-приложения, а `NavLink` дополнительно позволяет определять активную страницу и изменять стиль ссылки в зависимости от текущего маршрута.

Компонент `Header` возвращает JSX-разметку шапки сайта. Внешний тег `<header>` задает фон, цвет текста и тень для визуального отделения панели навигации от основного содержимого. Внутри находится тег `<nav>`, который содержит логотип и ссылки навигации. Компонент `Link` используется для перехода на главную страницу. Свойство `to="/"` указывает путь маршрута. При нажатии переход выполняется без перезагрузки страницы, так как маршрутизация происходит на стороне клиента. Для остальных пунктов меню используются `NavLink`. Их основное отличие заключается в поддержке проверки активности маршрута.

В параметрах `className` передается функция, а `React Router` автоматически передает объект с параметром `isActive`. Если ссылка соответствует текущему адресу страницы, применяется стиль активного пункта меню.

Таким образом, эти два компонента созданы и готовы к использованию в главном корневом `layout`, к которому мы перейдем позже, а пока создадим два оставшихся компонента — карточка меню и элемент корзины. Коды данных компонентов расположены на рисунках 4.6 и 4.7.

```

app > components > MenuCard.tsx > ...
...
1  import type { MenuItem } from "~/types";
2
...
3  interface Props {
4    item: MenuItem;
5    onAddToCart: (item: MenuItem) => void;
6  }
7
8  export default function MenuCard({ item, onAddToCart }: Props) {
9    return (
10     <div className="bg-white rounded-2xl shadow-md overflow-hidden hover:shadow-lg transition-shadow">
11       <img
12         src={item.image}
13         alt={item.name}
14         className="w-full h-48 object-cover"
15       />
16       <div className="p-5">
17         <div className="flex justify-between items-start mb-2">
18           <h3 className="font-bold text-lg">{item.name}</h3>
19           <span className="text-tom-thumb-700 font-bold">{item.price} ₽</span>
20         </div>
21         <p className="text-sm text-stone-500 mb-4">{item.description}</p>
22         <button
23           onClick={() => onAddToCart(item)}
24           className="w-full bg-tom-thumb-600 text-white py-2 rounded-xl hover:bg-tom-thumb-700 transition-colors"
25         >
26           В корзину
27         </button>
28       </div>
29     </div>
30   );
31 }

```

Рисунок 4.6 — MenuCard.tsx

```

app > components > CartItem.tsx > ...
...
1  import type { CartItem as CartItemType } from "~/types";
2
...
3  interface CartItemProps {
4    item: CartItemType;
5    onUpdateQuantity: (id: number, quantity: number) => void;
6  }
7
8  export default function CartItem({ item, onUpdateQuantity }: CartItemProps) {
9    const { menuItem, quantity } = item;
10
11    return (
12     <div className="bg-white rounded-xl p-4 flex items-center gap-4 shadow-sm">
13       <img
14         src={menuItem.image}
15         alt={menuItem.name}
16         className="w-20 h-20 object-cover rounded-lg"
17       />
18
19       <div className="flex-grow">
20         <h3 className="font-bold text-stone-800">{menuItem.name}</h3>
21         <p className="text-tom-thumb-700 font-medium">
22           {menuItem.price} ₽ × {quantity} = {menuItem.price * quantity} ₽
23         </p>
24       </div>
25
26       <div className="flex items-center gap-2">
27         <button
28           onClick={() => onUpdateQuantity(menuItem.id, quantity - 1)}
29           className="w-8 h-8 bg-stone-200 rounded-full hover:bg-stone-300 transition-colors flex items-center justify-center"
30         >
31           −
32         </button>
33         <span className="w-8 text-center font-medium">{quantity}</span>
34         <button
35           onClick={() => onUpdateQuantity(menuItem.id, quantity + 1)}
36           className="w-8 h-8 bg-stone-200 rounded-full hover:bg-stone-300 transition-colors flex items-center justify-center"
37         >
38           +
39         </button>
40       </div>
41     </div>
42   );
43 }

```

Рисунок 4.7 — CartItem.tsx

Файл `MenuCard.tsx` реализует компонент карточки блюда. Такой компонент используется для отображения информации о товаре или позиции меню в удобном и компактном виде. Карточка содержит изображение блюда, его название, описание, цену и кнопку добавления в корзину.

Для начала импортируется тип `MenuItem` из директории `types`. В отличие от предыдущих компонентов, здесь тип используется для строгого описания объекта блюда. Это позволяет TypeScript проверять наличие всех необходимых свойств, например названия, изображения, цены и описания.

Интерфейс `Props` описывает свойства компонента. Свойство `item` содержит объект блюда, а `onAddToCart` представляет собой функцию, которая вызывается при добавлении товара в корзину. Функция принимает объект `MenuItem`, благодаря чему компонент может передавать информацию о выбранном блюде в родительский компонент.

Внутри JSX-разметки создается карточка с белым фоном, скругленными углами и тенью. При наведении тень становится более выраженной, что создает визуальный эффект интерактивности. Изображение выводится через тег `<img>`.

Здесь `src` получает путь к изображению блюда, а `alt` используется для текстового описания изображения. Это важно как для доступности интерфейса, так и для корректной работы страницы при ошибке загрузки изображения. Название, цена и описание выводятся через свойства объекта `item`. React автоматически подставляет значения в JSX через фигурные скобки.

Главным элементом взаимодействия является кнопка «В корзину». При нажатии вызывается стрелочная функция, которая передает текущее блюдо в обработчик `onAddToCart`. Благодаря этому родительский компонент получает информацию о том, какой товар пользователь добавил в корзину.

`CartItem.tsx` отвечает за отображение отдельного товара в корзине. Компонент показывает информацию о выбранном блюде, количество товара и позволяет изменять это количество с помощью кнопок.

Здесь тоже импортируется тип `CartItem`, который переименовывается в `CartItemType` для избежания конфликта имен между типом и названием самого компонента. Интерфейс `CartItemProps` описывает свойства компонента. Свойство `item` содержит объект корзины, а `onUpdateQuantity` представляет функцию изменения количества товара. Она принимает идентификатор блюда и новое количество. Внутри компонента используется деструктуризация, которая позволяет получить доступ к вложенному объекту блюда и количеству товара без постоянного обращения через `item.menuItem` и `item.quantity`.

Разметка компонента во многом похожа на `MenuCard.tsx`: отображается изображение блюда, название и цена. Однако здесь появляется дополнительная логика расчета общей стоимости. React вычисляет выражение прямо внутри JSX и выводит итоговую сумму для выбранного количества товара.

Главное отличие компонента — наличие управления количеством. Для уменьшения и увеличения используются две кнопки. При нажатии вызывается функция `onUpdateQuantity`, которой передается идентификатор товара и новое значение количества. Такой подход позволяет изменять состояние корзины в родительском компоненте, сохраняя сам `CartItem` максимально простым и переиспользуемым.

Теперь, когда полностью изготовлены компоненты нашего интерфейса, можно сделать нашу обертку приложения в `root.tsx`. Но, перед этим, мы должны создать свой кастомный хук для работы с корзиной.

5 Создание кастомного хука

Напомню, что React позволяет создавать пользовательские хуки. Они позволяют вынести логику компонента (например, запросы к API, обработку форм, подписки) в отдельную переиспользуемую функцию, делая код чище и проще. Главное условие — указать в названии файла префикс `use`, чтобы данный файл считался хуком; он обязательно должен возвращать какое-то значение.

Также, внутри него может быть провайдер. «Провайдер» (Provider) — это специальный компонент из React Context API, который часто используют вместе с кастомными хуками для управления глобальным состоянием. Если обычный кастомный хук просто переиспользует логику (у каждого компонента будут свои независимые данные), то связка «Хук + Провайдер» позволяет разным компонентам работать с одними и теми же общими данными.

Создадим в папке `app` папку `hooks` и создадим в ней папку с названием `useCart.tsx`. На рисунке 5.1 расположена папка, а на рисунках 5.2 и 5.3 — сам код файла.

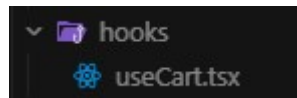


Рисунок 5.1 — hooks

```
app > hooks > useCart.tsx > ...  
  
You, 17 hours ago | 1 author (You)  
1 import { createContext, useContext, useMemo, useState } from "react";  
2 import type { ReactNode } from "react";  
3 import type { CartItem, MenuItem } from "~/types";  
4  
You, 17 hours ago | 1 author (You)  
5 interface CartContextValue {  
6   items: CartItem[];  
7   totalAmount: number;  
8   totalCount: number;  
9   addItem: (item: MenuItem) => void;  
10  updateQuantity: (id: number, newQty: number) => void;  
11  removeItem: (id: number) => void;  
12  clearCart: () => void;  
13 }  
14  
15 const CartContext = createContext<CartContextValue | null>(null);  
16  
17 export function CartProvider({ children }: { children: ReactNode }) {  
18   const [items, setItems] = useState<CartItem[]>([]);  
19  
20   const totalAmount = useMemo(  
21     () => items.reduce((sum, item) => sum + item.menuItem.price * item.quantity, 0),  
22     [items]  
23   );  
24  
25   const totalCount = useMemo(  
26     () => items.reduce((sum, item) => sum + item.quantity, 0),  
27     [items]  
28   );
```

Рисунок 5.2 — useCart.tsx (часть №1)

```

30   const addItem = (menuItem: MenuItem) => {
31     setItems((prev) => {
32       const existing = prev.find((item) => item.menuItem.id === menuItem.id);
33       if (existing) {
34         return prev.map((item) =>
35           item.menuItem.id === menuItem.id
36             ? { ...item, quantity: item.quantity + 1 }
37             : item
38         );
39       }
40       return [...prev, { menuItem, quantity: 1 }];
41     });
42   };
43
44   const updateQuantity = (id: number, newQty: number) => {
45     setItems((prev) =>
46       prev
47         .map((item) =>
48           item.menuItem.id === id ? { ...item, quantity: newQty } : item
49         )
50         .filter((item) => item.quantity > 0)
51     );
52   };
53
54   const removeItem = (id: number) => {
55     setItems((prev) => prev.filter((item) => item.menuItem.id !== id));
56   };
57
58   const clearCart = () => {
59     setItems([]);
60   };
61
62   return (
63     <CartContext.Provider
64       value={{ items, totalAmount, totalCount, addItem, updateQuantity, removeItem, clearCart }}
65     >
66       {children}
67     </CartContext.Provider>
68   );
69 }
70
71 export function useCart() {
72   const context = useContext(CartContext);
73   if (!context) {
74     throw new Error("useCart must be used within a CartProvider");
75   }
76   return context;
77 }

```

Рисунок 5.3 — useCart.tsx (часть №2)

Данный файл реализует контекст корзины для React-приложения. Его основная задача — организовать централизованное хранение данных о товарах в корзине и предоставить доступ к этим данным из любых компонентов приложения без необходимости вручную передавать свойства через несколько уровней вложенности.

Импортируются хуки `createContext`, `useContext`, `useMemo` и `useState`. Хук `useState` используется для хранения списка товаров корзины, `useMemo` — для оптимизации вычислений общей суммы и количества товаров, `createContext` создает контекст, а `useContext` позволяет получать доступ к данным контекста в других компонентах. Также импортируются типы `CartItem` и `MenuItem`, которые описывают структуру данных товаров и элементов корзины.

Интерфейс `CartContextValue` описывает структуру данных, которые будут доступны через контекст. Здесь находятся массив товаров `items`, общая сумма заказа `totalAmount`, общее

количество товаров `totalCount`, а также функции управления корзиной: добавление товара, изменение количества, удаление позиции и полная очистка корзины.

Далее создается сам контекст `const CartContext = createContext<CartContextValue | null>(null)`. Контекст в React представляет собой механизм передачи данных между компонентами без необходимости передавать их через `props` на каждом уровне. В данном случае контекст используется как глобальное хранилище корзины.

Главной частью файла является компонент `CartProvider`. Именно он выступает в роли провайдера контекста. Провайдер — это компонент-обертка, который предоставляет данные всем вложенным компонентам. Любой компонент, находящийся внутри `CartProvider`, может получить доступ к корзине через контекст. Свойство `children` содержит вложенные компоненты приложения. Это означает, что `CartProvider` обычно оборачивает большую часть интерфейса, например все приложение целиком. Благодаря этому данные корзины становятся доступны на любой странице: в меню, корзине, модальном окне оформления заказа и других компонентах.

Внутри провайдера создается состояние, где находится массив товаров корзины. Изначально корзина пустая. Затем с помощью `useMemo` вычисляются `totalAmount` и `totalCount`. Хук `useMemo` нужен для оптимизации производительности. Он предотвращает повторные вычисления при каждом рендере компонента и пересчитывает значения только тогда, когда изменяется массив `items`. `totalAmount` вычисляет общую стоимость всех товаров в корзине, а `totalCount` — общее количество товаров независимо от их типа.

Рассмотрим функции, которые здесь есть:

- `addItem` отвечает за добавление товара в корзину: сначала выполняется проверка, существует ли уже такой товар в массиве; если товар найден, увеличивается его количество; если нет — создается новая запись с количеством 1 — мы избегаем дублирования товаров в корзине;
- `updateQuantity` изменяет количество товара: она обновляет нужный элемент массива по идентификатору; дополнительно используется `filter`, который удаляет товары с количеством меньше или равным нулю;
- `removeItem` полностью удаляет товар по идентификатору, а `clearCart` очищает всю корзину сразу.

В блоке `return` компонент `CartProvider` возвращает `CartContext.Provider`. Именно этот элемент делает данные контекста доступными для вложенных компонентов. В свойство `value` передается объект со всеми данными и функциями корзины. React автоматически предоставляет эти значения всем дочерним компонентам внутри провайдера.

Последняя часть файла — пользовательский хук `useCart`. Он упрощает работу с контекстом и позволяет получать доступ к корзине через одну функцию. Вместо прямого использования `useContext(CartContext)` разработчик может просто вызвать `useCart()`. Здесь реализована проверка, которая предотвращает использование контекста вне `CartProvider`. Если компонент попытается обратиться к корзине без провайдера, приложение сразу выдаст ошибку.

Теперь переходим к `root.tsx`.

6 Корневой layout

Как уже было сказано ранее, данный файл (который уже есть внутри шаблона при инициализации проекта) является корневым компонентом приложения и отвечает за формирование общей структуры сайта. Именно здесь задается основной шаблон интерфейса, который будет использоваться на всех страницах приложения. Это главный макет приложения.

Здесь мы как раз таки и будем использовать разработанные ранее компоненты — шапка и подвал, и используем провайдер корзины.

Рассмотрим код на рисунке 6.1.

```
app > root.tsx > ...
You, 18 hours ago | 1 author (You)
1  import {
2    Links,
3    Meta,
4    Outlet,
5    Scripts,
6    ScrollRestoration,
7  } from "react-router";
8  import Header from "~/components/Header";
9  import Footer from "~/components/Footer";
10 import { CartProvider } from "~/hooks/useCart";
11 import "./app.css";
12
13 export default function RootLayout() {
14   return (
15     <html Lang="ru">
16       <head>
17         <meta charSet="utf-8" />
18         <meta name="viewport" content="width=device-width, initial-scale=1" />
19         <Meta />
20         <Links />
21       </head>
22       <body>
23         <CartProvider>
24           <div className="min-h-screen flex flex-col">
25             <Header />
26             <main className="flex-grow max-w-6xl mx-auto px-4 py-8 w-full">
27               <Outlet />
28             </main>
29             <Footer />
30           </div>
31         </CartProvider>
32
33         <ScrollRestoration />
34         <Scripts />
35       </body>
36     </html>
37   );
38 }
```

Рисунок 6.1 — root.tsx

Импортируем специальные компоненты из react-router: Links, Meta, Outlet, Scripts и ScrollRestoration.

Поговорим про ScrollRestoration. В веб-разработке, особенно при использовании SSR (Server-Side Rendering) и современных фреймворков, взаимодействие ScrollRestoration (восстановление позиции прокрутки) и гидратации (оживление статического HTML клиентским JavaScript) — это критически важный момент для пользовательского опыта (UX).

Гидратация (Hydration) — процесс, при котором клиентский JavaScript «привязывается» к статическому HTML-коду, полученному с сервера, делая его интерактивным, а ScrollRestoration — механизм браузера или фреймворка, который запоминает позицию скролла при уходе со страницы и возвращает её при возвращении назад. Главная проблема может возникнуть из-за асинхронности: браузер хочет восстановить позицию скролла сразу, а контент может быть ещё не готов (гидратация не завершена). Для этого мы используем данный механизм, чтобы избегать лишних ошибок.

Компонент Meta отвечает за подключение метаинформации страницы, например заголовков и SEO-данных. Links используется для подключения внешних ресурсов. Scripts автоматически подключает необходимые скрипты приложения. Компонент ScrollRestoration восстанавливает позицию прокрутки страницы при переходах между маршрутами, что улучшает пользовательский опыт. Также импортируются компоненты Header и Footer, которые уже используются как общие элементы интерфейса. Дополнительно подключается CartProvider — провайдер контекста корзины.

В RootLayout первыми идут язык и кодировка — если не выставить этих параметров, то мы не получим красивый интерфейс с правильными буквами, но поломанный текст. Указание кодировки и языка обязательно, не только для улучшения пользовательского опыта и правильной работе при распознавании текста, но и для SEO или защиты приложения.

CartProvider оборачивает основную структуру приложения и предоставляет доступ к данным корзины всем вложенным компонентам. Благодаря этому любой компонент внутри приложения может использовать корзину через хук useCart, независимо от уровня вложенности.

Внутри CartProvider располагается основная структура страницы. Используется контейнер с минимальной высотой экрана min-h-screen и flex-разметкой. Такой подход позволяет прижать футер к нижней части окна браузера даже при небольшом количестве контента. Компонент Header отображается в верхней части страницы, а Footer — в нижней. Между ними находится тег <main>, внутри которого используется компонент Outlet.

Outlet — один из ключевых элементов React Router. Он служит местом для отображения содержимого текущего маршрута. Когда пользователь переходит между страницами, React Router автоматически подставляет нужный компонент внутрь Outlet. Благодаря этому шапка и футер остаются неизменными, а меняется только основное содержимое страницы.

Также в файле подключается глобальный CSS-файл app.css, который содержит общие стили приложения, чтобы они применялись на все элементы интерфейса.

Теперь, когда почти все важные компоненты приложения готовы — перейдем к самим страницам приложения и настройке роутинга внутри него.

7 Роутинг в приложении

Для переходов между страницами в нашем приложении будет использоваться следующая структура, о которой было сказано ранее: папка со страницами и файл, который является соединительным узлом маршрутизации в приложении.

Сразу создадим все страницы в папке `routes` и опишем в `routes.ts`, как именно будет работать маршрутизация. На данном этапе, так как файлы пустые — могут возникать ошибки из-за того, что они не являются модулями. На данном этапе это не страшно, поскольку мы только настраиваем маршрутизацию и не заполняем файлы страниц содержимым. Это мы сделаем, а пока посмотрим на структуру папки `routes` на рисунке 7.1.

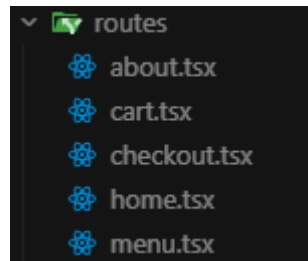


Рисунок 7.1 — routes

У нас есть:

- `about.tsx` — страница, на которой будет информация о заведении;
- `cart.tsx` — страница корзины, где можно управлять заказом и перейти на оплату;
- `checkout.tsx` — страница оплаты заказа;
- `home.tsx` — главная страница приложения;
- `menu.tsx` — страница с каталогом товаров и фильтрацией.

Теперь перейдем к файлу `routes.ts`. Он определяет, какие компоненты должны отображаться при переходе пользователя на определённые URL-адреса. Благодаря маршрутизации приложение работает как одностраничный сайт, где переходы между страницами происходят без полной перезагрузки браузера.

На рисунке 7.2 изображен код данного файла.

```
app > routes.ts > ...  
You, 20 hours ago | 1 author (You)  
1 import { type RouteConfig, index, route } from "@react-router/dev/routes";  
2  
3 export default [  
4   index("routes/home.tsx"),  
5   route("menu", "routes/menu.tsx"),  
6   route("cart", "routes/cart.tsx"),  
7   route("checkout", "routes/checkout.tsx"),  
8   route("about", "routes/about.tsx"),  
9 ] satisfies RouteConfig;
```

Рисунок 7.2 — routes.ts

В начале файла импортируются `RouteConfig`, `index` и `route` из библиотеки `@react-router/dev/routes`. Функция `route` используется для создания обычных маршрутов, а `index` — для настройки главной страницы приложения. Далее экспортируется массив маршрутов. Каждый элемент массива описывает отдельную страницу приложения и связывает URL-адрес с конкретным React-компонентом. `index` обозначает корневой маршрут приложения, то есть страницу, которая будет открываться по адресу `/`. В данном случае при открытии главной страницы будет загружаться компонент `home.tsx`. Все остальные компоненты определены через функцию `route`.

Нужно обратить внимание на конструкцию `satisfies RouteConfig`. Она используется в TypeScript для проверки соответствия структуры массива типу `RouteConfig`. Это позволяет обнаруживать ошибки в настройке маршрутов еще на этапе разработки и обеспечивает дополнительную типобезопасность.

8 Страницы приложения

Начнем с главной страницы — `home.tsx`. На данную страницу нас будет отсылать приложение, когда мы только его запустили.

На рисунке 8.1 представлен его код. Рассмотрим его подробнее.

```
app > routes > home.tsx > ...
You, 19 hours ago | 1 author (You)
1  import { Link } from "react-router";
2  import { restaurantInfo } from "~/data/restaurant";
3
4  export function meta() {
5    return [{ title: "HATK" }];
6  }
7
8  export default function HomePage() {
9    return (
10     <div className="text-center space-y-8">
11       <h1 className="text-5xl font-bold text-tom-thumb-900 mt-12">
12         {restaurantInfo.name}
13       </h1>
14       <p className="text-xl text-tom-thumb-700 max-w-2xl mx-auto">
15         Изысканная кухня в центре города.
16       </p>
17       <Link
18         to="/menu"
19         className="inline-block bg-tom-thumb-600 text-white px-8 py-4 rounded-xl text-lg hover:bg-tom-thumb-700 transition-colors"
20       >
21         Смотреть меню
22       </Link>
23     </div>
24   );
25 }
```

Рисунок 8.1 — `home.tsx`

Импортируем компонент `Link` из `react-router`, который используется для навигации между страницами без перезагрузки браузера. Также импортируется объект `restaurantInfo`, содержащий информацию о ресторане. Как и в предыдущих компонентах, данные берутся из отдельного файла, что делает приложение более удобным для поддержки и обновления. Отдельно определяется функция `meta`. Она отвечает за настройку метаданных страницы. В данном случае задается заголовок вкладки браузера.

Основной компонент `HomePage` возвращает JSX-разметку главной страницы. Внешний контейнер использует классы выравнивания текста и вертикальных отступов между элементами. Главный заголовок страницы выводится через `{restaurantInfo.name}`. React подставляет название ресторана из объекта данных. Ниже располагается краткое описание ресторана. Текст ограничивается по ширине и автоматически центрируется, что улучшает читаемость интерфейса.

Для перехода к меню используется компонент `Link`. Свойство `to="/menu"` указывает маршрут страницы меню. При нажатии происходит переход внутри React-приложения без обновления страницы браузера.

По логике, переходим к следующей странице — меню ресторана. Тут все тоже достаточно просто, т.к. мы уже имеем необходимые компоненты для его создания. На рисунках 8.2 и 8.3 представлен его код. Здесь главными особенностями являются фильтрация по категории блюд и добавление блюд в корзину.

app > routes > menu.tsx > ...

```
...
1  import { useState } from "react";
2  import { menuData } from "~/data/menu";
3  import MenuCard from "~/components/MenuCard";
4  import { useCart } from "~/hooks/useCart";
5  import type { MenuItem } from "~/types";
6
7  export function meta() {
8    return [{ title: "Меню | HATK" }];
9  }
10
11 export default function MenuPage() {
12   const categories = ["Все", "Закуски", "Основные блюда", "Десерты", "Напитки"];
13   const [activeCategory, setActiveCategory] = useState("Все");
14   const { totalCount, addItem } = useCart();
15
16   const filteredMenu = activeCategory === "Все"
17     ? menuData
18     : menuData.filter(item => item.category === activeCategory);
19
20   const addToCart = (item: MenuItem) => {
21     addItem(item);
22   };
23
```

Рисунок 8.2 — menu.tsx (часть №1)

```
24   return (
25     <div>
26       <div className="flex justify-between items-center mb-8">
27         <h1 className="text-4xl font-bold text-tom-thumb-900">Меню</h1>
28         <span className="bg-tom-thumb-100 text-tom-thumb-800 px-4 py-2 rounded-full">
29           {totalCount} блюд
30         </span>
31       </div>
32
33       <div className="flex gap-3 mb-8 flex-wrap">
34         {categories.map(cat => (
35           <button
36             key={cat}
37             onClick={() => setActiveCategory(cat)}
38             className={`px-5 py-2 rounded-full border transition-colors ${activeCategory === cat
39               ? "bg-tom-thumb-600 text-white border-tom-thumb-600"
40               : "bg-white text-tom-thumb-800 border-tom-thumb-200 hover:bg-tom-thumb-50"}
41           >
42             {cat}
43           </button>
44         ))}
45       </div>
46
47       <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
48         {filteredMenu.map(item => (
49           <MenuCard key={item.id} item={item} onAddToCart={addToCart} />
50         ))}
51       </div>
52     </div>
53   );
54 }
55
```

Рисунок 8.3 — menu.tsx (часть №2)

Первым делом импортируем хук `useState`, который используется для хранения текущей выбранной категории меню. Также импортируются данные `menuData`, компонент `MenuCard`, пользовательский хук `useCart` и тип `MenuItem`. Как и в предыдущих страницах, здесь присутствует функция `meta`, которая задает заголовок вкладки браузера. В данном случае отображается название страницы меню и название ресторана.

Внутри компонента `MenuPage` сначала создается массив категорий. Этот массив используется для формирования кнопок фильтрации. Далее с помощью `useState` создается состояние `activeCategory`. Оно хранит текущую выбранную категорию. По умолчанию устанавливается значение "Все", поэтому при первом открытии страницы отображаются все блюда.

Через пользовательский хук `useCart` компонент получает доступ к данным корзины. Здесь используется контекст корзины, созданный ранее через `CartProvider`. `totalCount` содержит общее количество товаров в корзине, а `addItem` — функцию добавления блюда.

Следующая часть реализует фильтрацию меню: если выбрана категория "Все", отображается полный список блюд; в противном случае используется метод `filter`, который оставляет только элементы с нужной категорией. Такой подход позволяет динамически изменять отображаемое меню без перезагрузки страницы.

Функция `addToCart` служит промежуточным обработчиком и передает выбранное блюдо в функцию `addItem` контекста корзины. В JSX-разметке сначала отображается заголовок страницы и счетчик товаров в корзине. Значение `totalCount` обновляется автоматически при изменении состояния корзины, так как данные поступают из контекста.

Ниже формируются кнопки категорий. Для этого используется метод `map`, который проходит по массиву `categories` и создает кнопку для каждой категории. При нажатии вызывается `setActiveCategory`, изменяющий текущее состояние фильтра.

Далее используются динамические классы: если категория активна, кнопка получает один набор стилей, иначе — другой. Благодаря этому пользователь визуально видит выбранный фильтр.

Последняя часть компонента отвечает за отображение карточек блюд. Здесь также используется `map`, который проходит по массиву `filteredMenu` и для каждого элемента создает компонент `MenuCard`. В `MenuCard` передаются три свойства: уникальный ключ `key`, объект блюда `item` и функция `onAddToCart`. Благодаря этому каждая карточка получает данные конкретного блюда и возможность взаимодействия с корзиной.

После меню будет логично рассмотреть страницу с логикой корзины, раз у нас уже есть логика добавления товаров в нее. На рисунках 8.4 и 8.5 рассмотрим код данной страницы поближе.

```

app > routes > cart.tsx > ...
You, 19 hours ago | 1 author (You)
1 import { Link } from "react-router";
2 import { useCart } from "~/hooks/useCart";
3
4 export function meta() {
5   return [{ title: "Корзина | НАТК" }];
6 }
7
8 export default function CartPage() {
9   const { items, totalAmount, updateQuantity } = useCart();
10
11   if (items.length === 0) {
12     return (
13       <div className="text-center py-20">
14         <h2 className="text-2xl font-bold mb-4">Корзина пуста</h2>
15         <Link to="/menu" className="text-tom-thumb-600 hover:underline text-lg">
16           Перейти в меню
17         </Link>
18       </div>
19     );
20   }
21

```

Рисунок 8.4 — cart.tsx (часть №1)

```

22   return (
23     <div className="max-w-2xl mx-auto">
24       <h1 className="text-3xl font-bold text-tom-thumb-900 mb-8">Корзина</h1>
25
26       {items.map(item => (
27         <div key={item.menuItem.id} className="bg-white rounded-xl p-4 mb-4 shadow-sm flex items-center gap-4">
28           <img src={item.menuItem.image} alt={item.menuItem.name} className="w-20 h-20 object-cover rounded-lg" />
29           <div className="flex-grow">
30             <h3 className="font-bold">{item.menuItem.name}</h3>
31             <p className="text-tom-thumb-700">{item.menuItem.price} Р</p>
32           </div>
33           <div className="flex items-center gap-2">
34             <button
35               onClick={() => updateQuantity(item.menuItem.id, item.quantity - 1)}
36               className="w-8 h-8 bg-stone-200 rounded-full"
37             >
38               −
39             </button>
40             <span className="w-8 text-center">{item.quantity}</span>
41             <button
42               onClick={() => updateQuantity(item.menuItem.id, item.quantity + 1)}
43               className="w-8 h-8 bg-stone-200 rounded-full"
44             >
45               +
46             </button>
47           </div>
48         </div>
49       )))
50
51     <div className="bg-stone-100 rounded-xl p-6 mt-6">
52       <div className="flex justify-between text-xl font-bold mb-4">
53         <span>Итого:</span>
54         <span>{totalAmount} Р</span>
55       </div>
56       <Link
57         to="/checkout"
58         className="block text-center w-full bg-tom-thumb-600 text-white py-3 rounded-xl text-lg hover:bg-tom-thumb-700 transition-colors"
59       >
60         Оформить заказ
61       </Link>
62     </div>
63   </div>
64 );
65 }

```

Рисунок 8.5 — cart.tsx (часть №2)

В начале файла импортируется компонент `Link` из `react-router` и пользовательский хук `useCart`. Как и в предыдущих страницах, через функцию `meta` задается заголовок вкладки браузера. Внутри компонента `CartPage` с помощью `useCart` получается доступ к данным корзины. Здесь `items` содержит массив товаров корзины, `totalAmount` — общую стоимость заказа, а `updateQuantity` используется для изменения количества товара. Все эти данные поступают из контекста, созданного через `CartProvider`.

Одной из важных частей компонента также является условительный рендеринг: если корзина пуста, компонент сразу возвращает отдельную разметку с сообщением для пользователя. Вместо списка товаров отображается текст «Корзина пуста» и ссылка на страницу меню. Если товары в корзине есть, отображается основная часть страницы. Сначала выводится заголовок, после чего с помощью метода `map` происходит перебор массива `items`. Для каждого элемента создается отдельный блок товара.

Внутри карточки отображаются изображение блюда, его название и цена. Структура во многом похожа на компонент `CartItem.tsx`, однако здесь разметка встроена непосредственно в страницу, а не вынесена в отдельный компонент. Для изменения количества используются две кнопки. При нажатии вызывается функция `updateQuantity`, которая получает идентификатор товара и новое количество. Таким образом пользователь может увеличивать или уменьшать количество товара прямо в корзине. Обновление интерфейса происходит автоматически благодаря изменению состояния в контексте.

После списка товаров отображается блок итоговой информации. Здесь выводится общая сумма заказа через `totalAmount`. Значение пересчитывается автоматически при изменении содержимого корзины, так как вычисляется внутри контекста через `useMemo`.

Ниже располагается ссылка на страницу оформления заказа, что позволяет перейти на страницу оплаты без перезагрузки страницы. Конечно, в реальных приложениях это работает по-другому, так как важно перебросить пользователя на страницу безопасной оплаты и обычно там используются сторонние сервисы для этого, но у нас будет только имитация процесса оплаты заказа, поэтому этого будет достаточно.

Перейдем к крайней важной странице нашего приложения — имитация оплаты. На рисунках с 8.6 по 8.10 показан код данной страницы.

```
app > routes > checkout.tsx > ...
You, 1 minute ago | 1 author (You)
1  import { useState, type SubmitEvent } from "react";
2  import { Link, useNavigate } from "react-router";
3  import { useCart } from "~/hooks/useCart";
4  import Modal from "~/components/ui/Modal";
5  import Button from "~/components/ui/Button";
6
7  export function meta() {
8    return [{ title: "Оформление заказа | HATK" }];
9  }
10
11 export default function CheckoutPage() {
12   const { items, totalAmount, clearCart } = useCart();
13   const navigate = useNavigate();
14
15   const [name, setName] = useState("");
16   const [phone, setPhone] = useState("");
17   const [comment, setComment] = useState("");
18   const [paymentMethod, setPaymentMethod] = useState<"card" | "cash">("card");
19   const [isModalOpen, setIsModalOpen] = useState(false);
20   const [isProcessing, setIsProcessing] = useState(false);
21
22   if (items.length === 0) {
23     return (
24       <div className="text-center py-20">
25         <h2 className="text-2xl font-bold text-stone-700 mb-4">
26           Нечего оформлять
27         </h2>
28         <Link to="/menu" className="text-tom-thumb-600 hover:underline text-lg">
29           Перейти в меню
30         </Link>
31       </div>
32     );
33   }
```

Рисунок 8.6 — checkout.tsx (часть №1)

```

35     const handleSubmit = (e: SubmitEvent<HTMLFormElement>) => {
36         e.preventDefault();
37
38         if (!name.trim() || !phone.trim()) {
39             alert("Заполните имя и телефон");
40             return;
41         }
42
43         setIsProcessing(true);
44
45         setTimeout(() => {
46             setIsProcessing(false);
47             setIsModalOpen(true);
48         }, 2000);
49     };
50
51     const handleCloseModal = () => {
52         setIsModalOpen(false);
53         clearCart();
54         navigate("/");
55     };
56

```

Рисунок 8.7 — checkout.tsx (часть №2)

```

57     return (
58         <div className="max-w-2xl mx-auto">
59             <h1 className="text-4xl font-bold text-tom-thumb-900 mb-8 text-center">
60                 Оформление заказа
61             </h1>
62
63             <form onSubmit={handleSubmit} className="space-y-6">
64                 <div>
65                     <label className="block text-stone-700 font-medium mb-2">
66                         Ваше имя *
67                     </label>
68                     <input
69                         type="text"
70                         value={name}
71                         onChange={(e) => setName(e.target.value)}
72                         required
73                         className="w-full border border-stone-300 rounded-xl px-4 py-3 focus:outline-none focus:ring-2 focus:ring-tom-thumb-400"
74                         placeholder="Иван Иванов"
75                     />
76                 </div>
77
78                 <div>
79                     <label className="block text-stone-700 font-medium mb-2">
80                         Телефон *
81                     </label>
82                     <input
83                         type="tel"
84                         value={phone}
85                         onChange={(e) => setPhone(e.target.value)}
86                         required
87                         className="w-full border border-stone-300 rounded-xl px-4 py-3 focus:outline-none focus:ring-2 focus:ring-tom-thumb-400"
88                         placeholder="+7 (999) 123-45-67"
89                     />
90                 </div>

```

Рисунок 8.8 — checkout.tsx (часть №3)

```

92 <div>
93   <label className="block text-stone-700 font-medium mb-2">
94     Комментарий к заказу
95   </label>
96   <textarea
97     value={comment}
98     onChange={(e) => setComment(e.target.value)}
99     className="w-full border border-stone-300 rounded-xl px-4 py-3 focus:outline-none focus:ring-2 focus:ring-tom-thumb-400"
100     rows={3}
101     placeholder="Пожелания, аллергии..."
102   />
103 </div>
104
105 <div>
106   <label className="block text-stone-700 font-medium mb-2">
107     Способ оплаты
108   </label>
109   <div className="flex gap-6">
110     <label className="flex items-center gap-2 cursor-pointer">
111       <input
112         type="radio"
113         value="card"
114         checked={paymentMethod === "card"}
115         onChange={() => setPaymentMethod("card")}
116         className="accent-tom-thumb-600"
117       />
118       Картой онлайн
119     </label>
120     <label className="flex items-center gap-2 cursor-pointer">
121       <input
122         type="radio"
123         value="cash"
124         checked={paymentMethod === "cash"}
125         onChange={() => setPaymentMethod("cash")}
126         className="accent-tom-thumb-600"
127       />
128       Наличными
129     </label>
130   </div>
131 </div>

```

Рисунок 8.9 — checkout.tsx (часть №4)

```

133     <div className="bg-stone-100 rounded-2xl p-5">
134       <h3 className="font-bold text-stone-800 mb-3">Ваш заказ:</h3>
135       {items.map((item) => (
136         <div
137           key={item.menuItem.id}
138           className="flex justify-between text-sm text-stone-600 py-1"
139         >
140           <span>
141             {item.menuItem.name} {item.quantity}
142           </span>
143           <span>{item.menuItem.price * item.quantity} Р</span>
144         </div>
145       ))}
146       <div className="border-t border-stone-300 mt-3 pt-3 flex justify-between font-bold text-lg">
147         <span>Итого:</span>
148         <span className="text-tom-thumb-700">{totalAmount} Р</span>
149       </div>
150     </div>
151
152     <Button
153       type="submit"
154       disabled={isProcessing}
155       className="w-full py-4 text-lg"
156     >
157       {isProcessing ? "Обработка платежа..." : "Оплатить заказ"}
158     </Button>
159   </form>
160
161   <Modal
162     isOpen={isModalOpen}
163     onClose={handleCloseModal}
164     title="Заказ оформлен!"
165   >
166     <div className="text-center py-4">
167       <p className="text-lg text-stone-700 mb-2">
168         Спасибо, {name}!
169       </p>
170       <p className="text-stone-500 mb-6">
171         Ваш заказ на сумму {totalAmount} Р принят.
172         Мы свяжемся с вами по телефону {phone}.
173       </p>
174       <Button onClick={handleCloseModal} className="w-full">
175         На главную
176       </Button>
177     </div>
178   </Modal>
179 </div>
180 );
181 }

```

Рисунок 8.10 — checkout.tsx (часть №5)

В начале импортируются хуки `useState` и тип `SubmitEvent`. Хук `useState` используется для хранения данных формы и состояний интерфейса, а `SubmitEvent` позволяет типизировать событие отправки формы в TypeScript. Также импортируются `Link` и `useNavigate` из `react-router`. `Link` уже использовался ранее для навигации между страницами, а `useNavigate` позволяет выполнять программный переход между маршрутами через код.

Через пользовательский хук `useCart` компонент получает доступ к корзине. Здесь `items` содержит товары заказа, `totalAmount` — итоговую стоимость, а `clearCart` используется для очистки корзины после успешного оформления заказа. Отдельно вызывается `useNavigate`, который возвращает функцию навигации. Она понадобится позже для автоматического перехода пользователя на главную страницу.

Далее создается несколько состояний через `useState`. В отличие от предыдущих компонентов, здесь состояние используется достаточно активно. Отдельно хранятся имя пользователя, телефон, комментарий к заказу, способ оплаты, состояние модального окна и состояние обработки платежа.

Для способа оплаты используется строгая типизация. Это означает, что значение может быть только `"card"` или `"cash"`.

Как и на странице корзины, здесь присутствует условительный рендеринг. Если корзина пуста, пользователь увидит сообщение и ссылку на меню. Это предотвращает оформление заказа без товаров.

Главная логика оформления заказа находится в функции `handleSubmit`. Она вызывается при отправке формы. Этот метод отменяет стандартное поведение HTML-формы, при котором браузер перезагружает страницу после отправки. Далее выполняется простая проверка заполнения имени и телефона. Метод `trim()` удаляет лишние пробелы по краям строки. Если поля пустые, отображается сообщение через `alert`. После проверки включается состояние обработки заказа. Оно используется для блокировки кнопки и отображения текста «Обработка платежа...».

Затем используется `setTimeout`, который имитирует задержку обработки платежа. Через две секунды обработка завершается и открывается модальное окно подтверждения заказа. Функция `handleCloseModal` отвечает за закрытие модального окна после подтверждения заказа. Здесь выполняются сразу несколько действий: закрывается модальное окно, очищается корзина и выполняется переход на главную страницу через `navigate("/")`.

Основная часть JSX содержит форму оформления заказа. Поля ввода работают как управляемые компоненты `React`. Значение каждого поля связано с состоянием через `value`, а изменения обрабатываются через `onChange`. При каждом вводе символа состояние автоматически обновляется, а `React` повторно отображает актуальное значение. Для выбора способа оплаты используются радиокнопки (`radio`). Состояние `paymentMethod` определяет, какой вариант выбран в данный момент.

Ниже отображается блок с итогами заказа. С помощью `map` перебирается массив товаров корзины и выводится название блюда, количество и стоимость. Итоговая сумма отображается через `totalAmount`. Для кнопки отправки используется ранее созданный компонент `Button`.

Отдельного внимания заслуживает использование компонента `Modal`. Модальное окно отображается только тогда, когда `isModalOpen` равно `true`. Внутри модального окна передается динамическая информация: имя пользователя, сумма заказа и номер телефона, чтобы позже отобразить сообщение о том, что с пользователем свяжутся по заказу.

Теперь осталась крайняя страница — о ресторане. Здесь мы можем сделать любую страницу, какую душе угодно. Было решено сделать небольшой блок и фотографию в ресторане. На рисунке 8.11 изображен код данной страницы.

```
app > routes > about.tsx > ...
You 9 seconds ago | 1 author (You)
1 import { restaurantInfo } from "~/data/restaurant";
2 import restaurantImage from "~/assets/restaurant.avif";
3
4 export function meta() {
5   return [{ title: "О нас | HATK" }];
6 }
7
8 export default function AboutPage() {
9   return (
10     <div className="max-w-4xl mx-auto space-y-8">
11       <section className="bg-tom-thumb-50 rounded-3xl p-8 shadow-sm">
12         <h1 className="text-4xl font-bold text-tom-thumb-900 mb-4">О нас</h1>
13         <p className="text-lg text-tom-thumb-700 leading-relaxed">
14           HATK – ресторан с уютной атмосферой и современной европейской кухней. Мы готовим блюда из свежих продуктов, уделяем внимание качеству и создаём тёплую обстановку для гостей.
15         </p>
16       </section>
17
18       <section className="grid gap-6 md:grid-cols-2">
19         <div className="rounded-3xl bg-white p-8 shadow-sm">
20           <h2 className="text-2xl font-semibold text-tom-thumb-900 mb-3">Наши преимущества</h2>
21           <ul className="space-y-3 text-tom-thumb-700">
22             <li>Свежие ингредиенты и авторские рецепты</li>
23             <li>Быстрое обслуживание и внимательный персонал</li>
24             <li>Удобное расположение в центре города</li>
25             <li>Уютный интерьер и вечерняя атмосфера</li>
26           </ul>
27         </div>
28         <div className="rounded-3xl bg-white p-8 shadow-sm flex items-center justify-center">
29           <img
30             src={restaurantImage}
31             alt="Ресторан HATK"
32             className="w-full h-auto rounded-2xl object-cover"
33           />
34         </div>
35       </section>
36     </div>
37   );
38 }
```

Рисунок 8.11 - `about.tsx`

9 Запуск приложения

Остался последний шаг разработки — создать сборку приложения и запустить его. Для начала нам нужно запустить команду `npm run build`. На рисунке 9.1 изображен примерный результат запуска данной команды.

```
PS D:\projects\restaurant-app> npm run build

> build
> react-router build

vite v8.0.10 building client environment for production...
✓ 41 modules transformed.
computing gzip size...
build/client/.vite/manifest.json          4.80 kB | gzip: 0.86 kB
build/client/assets/bread_with_caviar-NTC3dAPN.avif 49.28 kB
build/client/assets/chips_with_meat-B5mSEeZ4.avif 53.63 kB
build/client/assets/red_fish-ChnxtRSP.avif 54.22 kB
build/client/assets/pumpkin_soup-CgMpq_8f.avif 67.05 kB
build/client/assets/avocado_toast-mXUdx1Ms.avif 73.37 kB
build/client/assets/restaurant-CUMxQj1D.avif 75.82 kB
build/client/assets/sweet_toast-CxOpqukw.avif 82.00 kB
build/client/assets/taco-BEBwpXDN.avif 83.98 kB
build/client/assets/pasta-BEMwkIfz.avif 98.09 kB
build/client/assets/greek_salad-Cspnww04.avif 139.83 kB
build/client/assets/oysters-BxvzmlU4.avif 149.95 kB
build/client/assets/set_of_starters-BEwNpxKR.avif 168.81 kB
build/client/assets/vegan_beans-B5mPam-z.avif 204.36 kB
build/client/assets/root-DHrbM170.css      16.39 kB | gzip: 3.98 kB
build/client/assets/restaurant-Bv8vH2TN.js    0.14 kB | gzip: 0.16 kB
build/client/assets/home-D-uQ_Ns4.js         0.70 kB | gzip: 0.46 kB
build/client/assets/useCart-ixYHMB3.js       0.83 kB | gzip: 0.46 kB
build/client/assets/about-y9eywzKn.js        1.83 kB | gzip: 0.90 kB
build/client/assets/cart-WbCvj-XS.js          1.98 kB | gzip: 0.84 kB
build/client/assets/root-Ch4iQ2px.js         2.00 kB | gzip: 0.74 kB
build/client/assets/menu-B_0nrrJ9.js         4.96 kB | gzip: 1.76 kB
build/client/assets/checkout-RtHmrDwB.js     5.23 kB | gzip: 1.87 kB
build/client/assets/jsx-runtime-y489x0t1.js 127.65 kB | gzip: 42.12 kB
build/client/assets/entry.client-CtC_gL5g.js 186.50 kB | gzip: 58.90 kB

✓ built in 261ms
vite v8.0.10 building SSR environment for production...
✓ 20 modules transformed.

✓ 2 assets cleaned from React Router server build.
build\client\assets\restaurant-CUMxQj1D.avif
build\server\assets\server-build-DHrbM170.css

computing gzip size...
build/server/.vite/manifest.json          0.41 kB | gzip: 0.20 kB
build/server/assets/restaurant-CUMxQj1D.avif 75.82 kB
build/server/assets/server-build-DHrbM170.css 16.39 kB | gzip: 3.98 kB
build/server/index.js                     32.80 kB | gzip: 7.65 kB

✓ built in 89ms
```

Рисунок 9.1 — Результат запуска `npm run build`

Команда "build": "react-router build" в файле `package.json` предназначена для сборки React-приложения для продакшена (продуктовой среды) с использованием современных инструментов React Router (React Router интегрируется с Vite). После этого в папке проекта должна создаваться папка `build`, содержимое которой показано на рисунке 9.2.

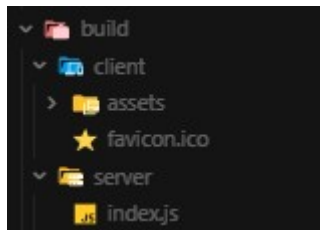


Рисунок 9.2 — Папка build

В папку build попадает полностью готовый к работе (скомпилированный) проект. Поскольку современный React Router v7 может работать и как обычное SPA, и как полноценный Full-stack фреймворк, содержимое папки разделено на части:

- client/assets/: Здесь лежат ваши оптимизированные JS-бандлы, CSS-стили и медиафайлы (картинки, шрифты); имена файлов обычно содержат хеш (например, index-a1b2c3d4.js) для эффективного кэширования в браузере;

- server – появляется, если вы используете функции SSR (Server-Side Rendering); внутри находится скомпилированный код ваших loader и action функций, а также логика рендеринга HTML на стороне сервера; этот код запускается в среде Node.js или на Edge-платформах.

Код в build не предназначен для чтения человеком. Он минифицирован (сжат) и обфусцирован (запутан) для максимальной производительности. Эту папку не нужно заливать в Git. Она автоматически пересоздается при каждом запуске команды сборки.

Теперь можно запустить команду `npm run start`. Что происходит при ее запуске, показано на рисунке 9.3.

```
PS D:\projects\restaurant-app> npm run start

> start
> react-router-serve ./build/server/index.js

[react-router-serve] http://localhost:3000 (http://100.66.47.129:3000)
```

Рисунок 9.3 — Результат запуска `npm run start`

Как можно заметить, приложение запущено на хосте с портом 3000 из сборки нашего проекта. Теперь можно перейти по URL и посмотреть, что получилось.

На рисунках ниже показано то, как в результате может выглядеть приложение.

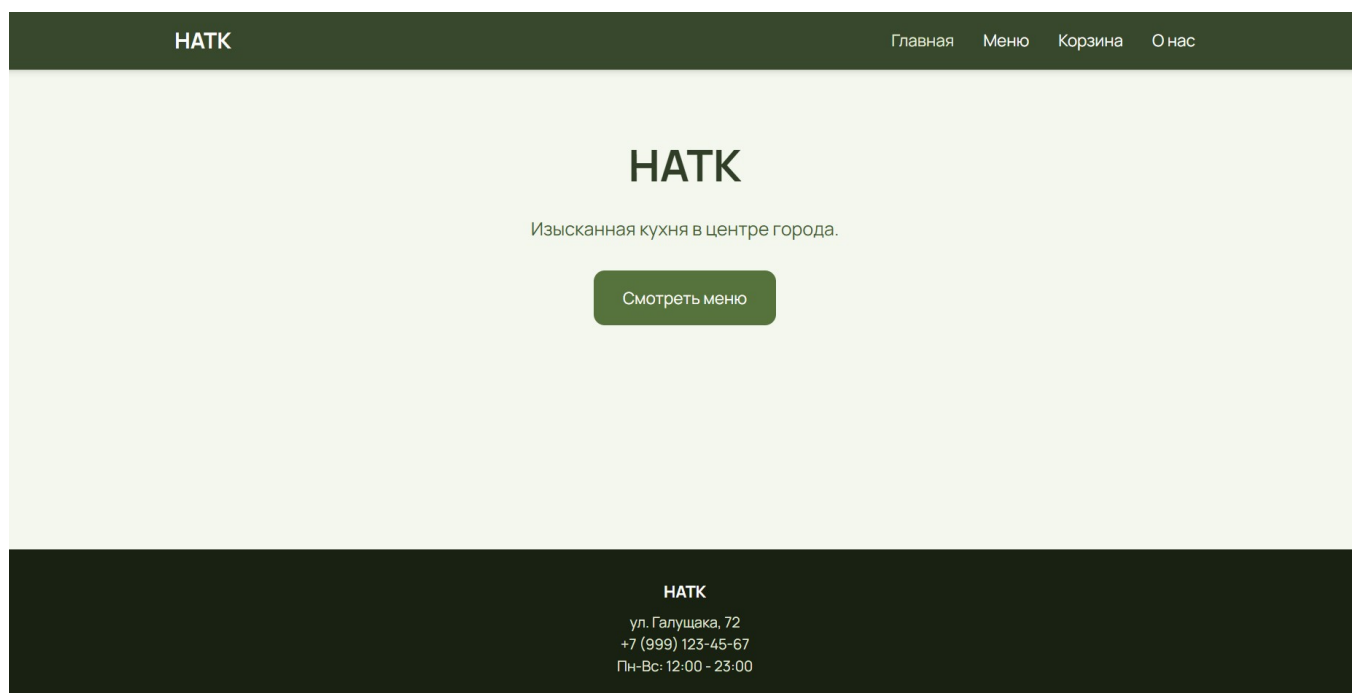


Рисунок 9.4 — Главная страница

Меню

0 блюд

- Все
- Закуски
- Основные блюда
- Десерты
- Напитки



Тост с авокадо и яйцом 250 Р

Тост с авокадо и яйцом с листиками базилика и оливковым маслом

В корзину



Багет с черной икрой 250 Р

Багет с черной икрой и кусочками сот с медом

В корзину



Чипсы и мясная нарезка 250 Р

Чипсы и мясная нарезка с сыром и оливками

В корзину



Греческий салат 250 Р

Греческий салат с оливками, помидорами, красным болгарским перцем и орегано

В корзину



Устрицы 250 Р

Устрицы с лимоном и горчицей

В корзину



Паста 250 Р

Паста с мясным соусом и базиликом

В корзину



Тыквенный суп 250 Р

Тыквенный суп с семенами тыквы и сливочным сыром

В корзину



Красная рыба 250 Р

Красная рыба с овощами и базиликом

В корзину



Сет закусок 250 Р

Всего понемногу

В корзину

Рисунок 9.5 — Страница с меню

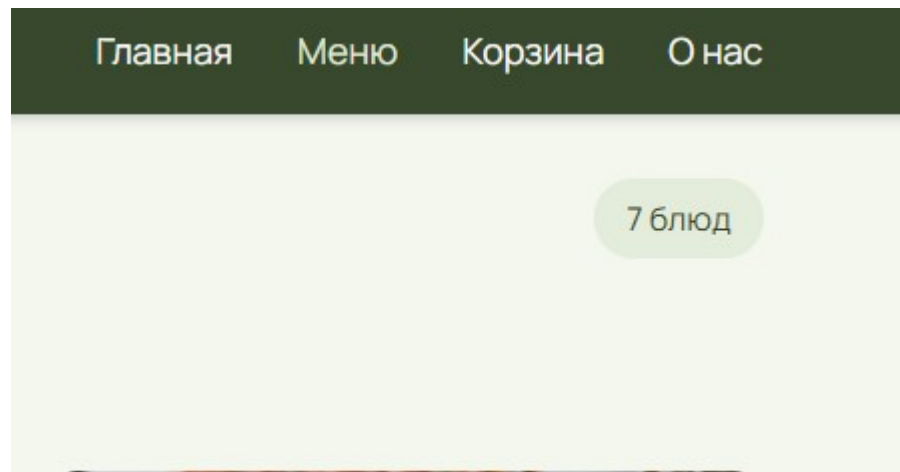


Рисунок 9.6 — Добавление блюд в корзину

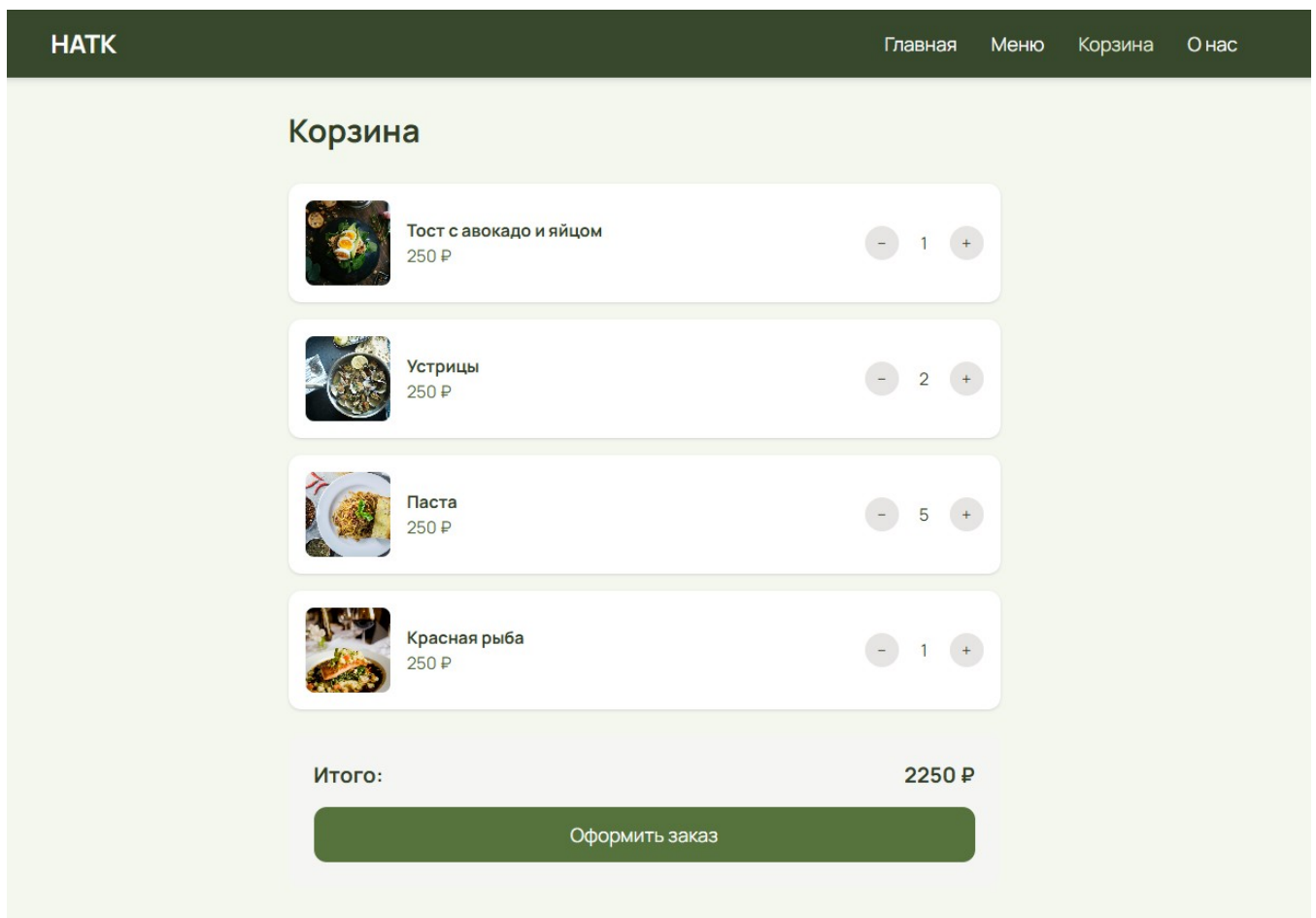


Рисунок 9.7 — Страница заказа и изменение количества позиций

Оформление заказа

Ваше имя *

Иван Иванов

Телефон *

+7 913 123 45 67

Комментарий к заказу

Хочу в коробке с миньонами

Способ оплаты

☐ Картой онлайн ☒ Наличными

Ваш заказ:

Тост с авокадо и яйцом × 1	250 Р
Устрицы × 2	500 Р
Паста × 5	1250 Р
Красная рыба × 1	250 Р

Итого: 2250 Р

Оплатить заказ

Рисунок 9.8 — Страница с формой оформления заказа

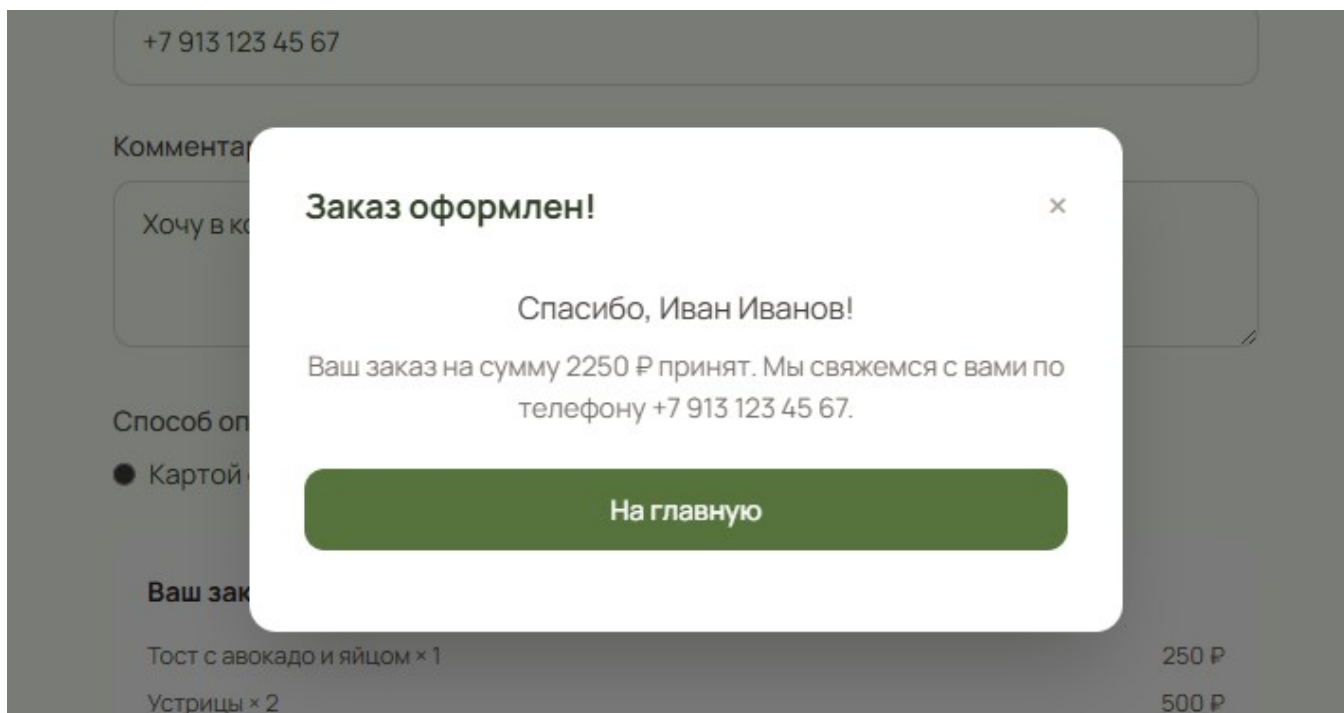


Рисунок 9.9 — Модальное окно после нажатия кнопки оформления заказа

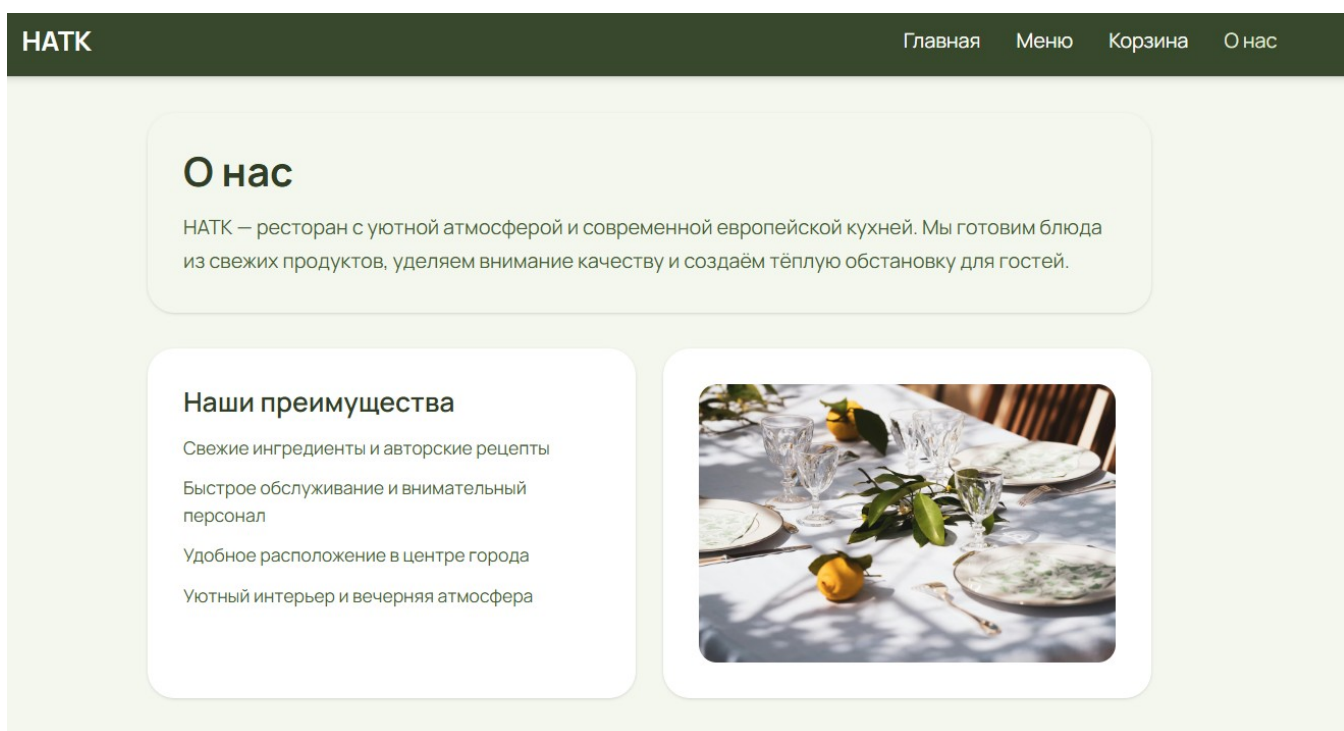


Рисунок 9.10 — Страница с информацией о ресторане

7 Отчет к работе

Отчетом к практическому занятию является:

- прикрепленная ссылка на GitHub-репозиторий;
- комментарии к коду (во всех файлах, кроме файлов документации, .gitignore, и тех, которые создаются автоматически или не содержат функционала);
- результат работы программы на видео/фото (загруженный либо на облачное хранилище, либо в отдельную папку в репозитории).

Заключение

В ходе данного практического занятия мы освоили процесс создания многостраничного веб-приложения на React с использованием современных инструментов разработки, таких как Vite, TypeScript и Tailwind CSS, реализовали механизм клиентской маршрутизации для мгновенных переходов между разделами и спроектировали систему глобального управления состоянием корзины через кастомный хук и React Context API.